

Big data architectures

A big data architecture manages the ingestion, processing, and analysis of data that's too large or complex for traditional database systems. The threshold for entering the realm of big data varies among organizations, depending on their tools and user capabilities. Some organizations manage hundreds of gigabytes of data, and other organizations manage hundreds of terabytes. As tools for working with big datasets evolve, the definition of big data shifts from focusing solely on data size to emphasizing the value derived from advanced analytics. Although these types of scenarios tend to have large amounts of data.

Over the years, the data landscape has changed. What you can do, or are expected to do, with data has changed. The cost of storage has fallen dramatically, while the methods for data collection continue to expand. Some data arrives at a rapid pace and requires continuous collection and observation. Other data arrives more slowly, but in large chunks, and often in the form of decades of historical data. You might encounter an advanced analytics problem or a problem that requires machine learning to solve. Big data architectures strive to solve these challenges.

Big data solutions typically involve one or more of the following types of workloads:

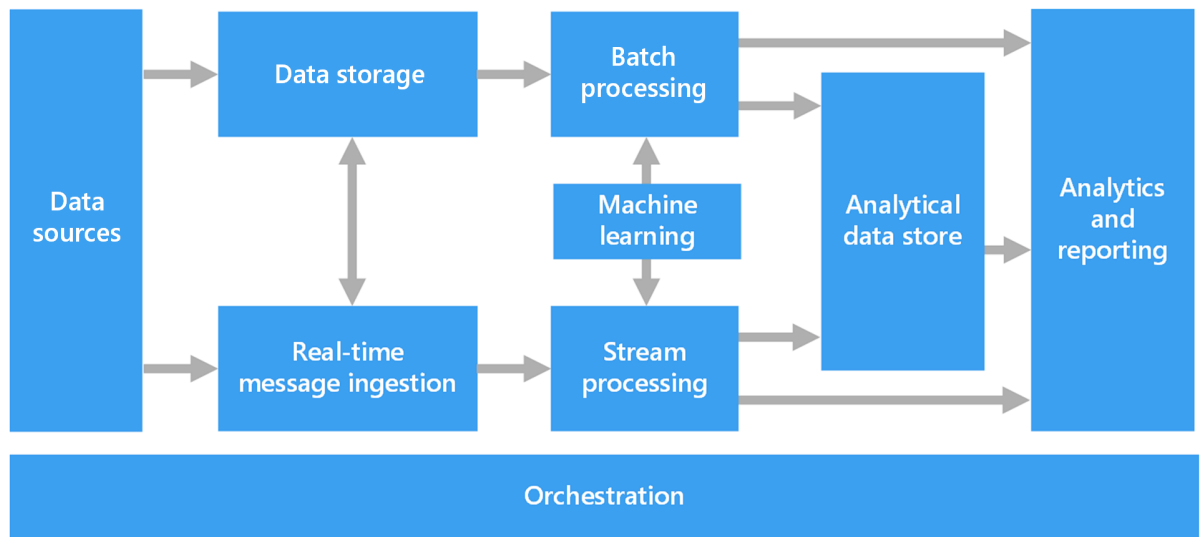
- Batch processing of big data sources at rest
- Real-time processing of big data in motion
- Interactive exploration of big data
- Predictive analytics and machine learning

Consider big data architectures when you need to do the following tasks:

- Store and process data in volumes that are too large for a traditional database
- Transform unstructured data for analysis and reporting
- Capture, process, and analyze unbounded streams of data in real time or with low latency

Components of a big data architecture

The following diagram shows the logical components of a big data architecture. Individual solutions might not contain every item in this diagram.



Most big data architectures include some or all of the following components:

- **Data sources:** All big data solutions start with one or more data sources. Examples include:
 - Application data stores, such as relational databases.
 - Static files that applications produce, such as web server log files.
 - Real-time data sources, such as Internet of Things (IoT) devices.
- **Data storage:** Data for batch processing operations is typically stored in a distributed file store that can hold high volumes of large files in various formats. This kind of store is often called a *data lake*. Options for implementing this storage include Azure Data Lake Storage, blob containers in Azure Storage, or OneLake in Microsoft Fabric.
- **Batch processing:** The datasets are large, so a big data solution often processes data files by using long-running batch jobs to filter, aggregate, and otherwise prepare data for analysis. Usually these jobs involve reading source files, processing them, and writing the output to new files. You can use the following options:
 - Use Python, Scala, or SQL language in Azure Databricks notebooks.
 - Use Python, Scala, or SQL language in Fabric notebooks.
- **Real-time message ingestion:** If the solution includes real-time sources, the architecture must capture and store real-time messages for stream processing. For example, you can have a simple data store that collects incoming messages for processing. However, many solutions need a message ingestion store to serve as a buffer for messages, and to support scale-out processing, reliable delivery, and other message queuing semantics. This part of a

streaming architecture is often referred to as *stream buffering*. Options include Azure Event Hubs, Azure IoT Hub, and Kafka.

- **Stream processing:** After the solution captures real-time messages, it must process them by filtering, aggregating, and preparing the data for analysis. The processed stream data is then written to an output sink.
 - You can use open-source Apache streaming technologies, like Spark Streaming, streaming technologies in Azure Databricks.
 - Azure Functions is a serverless compute service that can run event-driven code, which is ideal for lightweight stream processing tasks.
 - Fabric supports real-time data processing by using event streams and Spark processing.
- **Analytical data store:** Many big data solutions prepare data for analysis and then serve the processed data in a structured format that analytical tools can query. The analytical data store that serves these queries can be a Kimball-style relational data warehouse. Most traditional business intelligence (BI) solutions use this type of data warehouse. Alternatively, you can present the data through a low-latency NoSQL technology, such as HBase, or an interactive Hive database that provides a metadata abstraction over data files in the distributed data store.
 - Fabric provides various data stores, including SQL databases, data warehouses, lakehouses, and eventhouses. These tools can serve data for analysis.
 - Azure provides other analytical data stores, such as Azure Databricks, Azure Data Explorer, Azure SQL Database, and Azure Cosmos DB.
- **Analytics and reporting:** Most big data solutions strive to provide insights into the data through analysis and reporting. To empower users to analyze the data, the architecture might include a data modeling layer, such as a multidimensional online analytical processing cube or tabular data model in Azure Analysis Services. It might also support self-service BI by using the modeling and visualization technologies in Power BI or Excel.

Data scientists or data analysts can also analyze and report through interactive data exploration. For these scenarios, many Azure services support analytical notebooks, such as Jupyter, to enable these users to use their existing skills with Python or Microsoft R. For large-scale data exploration, you can use Microsoft R Server, either standalone or with Spark. You can also use Fabric to edit data models, which provides flexibility and efficiency for data modeling and analysis.

- **Orchestration:** Most big data solutions consist of repeated data processing operations that are encapsulated in workflows. The operations do the following tasks:

- Transform source data
- Move data between multiple sources and sinks
- Load the processed data into an analytical data store
- Push the results directly to a report or dashboard

To automate these workflows, use an orchestration technology such as Azure Data Factory, Fabric, or Apache Oozie and Apache Sqoop.

Lambda architecture

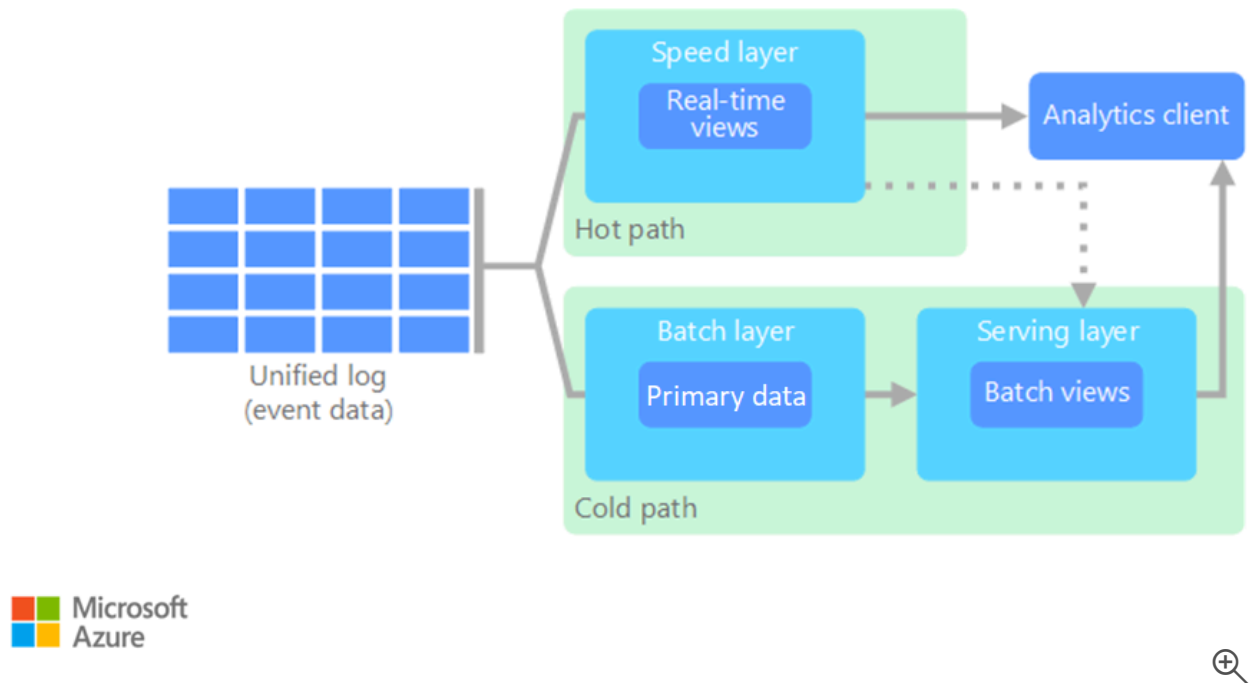
When you work with large datasets, it can take a long time to run the type of queries that clients need. These queries can't be performed in real time, and they often require distributed processing algorithms such as [MapReduce](#) that operate in parallel across the entire dataset. The query results are stored separately from the raw data and used for further querying.

One drawback to this approach is that it introduces latency. If processing takes a few hours, a query might return results that are several hours old. Ideally, you should get some results in real time, potentially with a loss of accuracy, and combine these results with the results from batch analytics.

The **Lambda architecture** addresses this problem by creating two paths for data flow. All data that comes into the system goes through the following two paths:

- A **batch layer** (cold path) stores all the incoming data in its raw form and performs batch processing on the data. The result of this processing is stored as a batch view.
- A **speed layer** (hot path) analyzes data in real time. This layer is designed for low latency, at the expense of accuracy.

The batch layer feeds into a **serving layer** that indexes the batch view for efficient querying. The speed layer updates the serving layer with incremental updates based on the most recent data.



Data that flows into the hot path must be processed quickly because of latency requirements that the speed layer imposes. Quick processing ensures that data is ready for immediate use but can introduce inaccuracy. For example, consider an IoT scenario where numerous temperature sensors send telemetry data. The speed layer might process a sliding time window of the incoming data.

Data that flows into the cold path isn't subject to the same low latency requirements. The cold path provides high accuracy computation across large datasets but can take a long time.

Eventually, the hot and cold paths converge at the analytics client application. If the client needs to display timely, yet potentially less accurate data in real time, it acquires its result from the hot path. Otherwise, the client selects results from the cold path to display less timely but more accurate data. In other words, the hot path has data for a relatively small window of time, after which the results can be updated with more accurate data from the cold path.

The raw data that's stored at the batch layer is immutable. Incoming data is appended to the existing data, and the previous data isn't overwritten. Changes to the value of a particular datum are stored as a new time-stamped event record. Time-stamped event records allow for recomputation at any point in time across the history of the data collected. The ability to recompute the batch view from the original raw data is important because it enables the creation of new views as the system evolves.

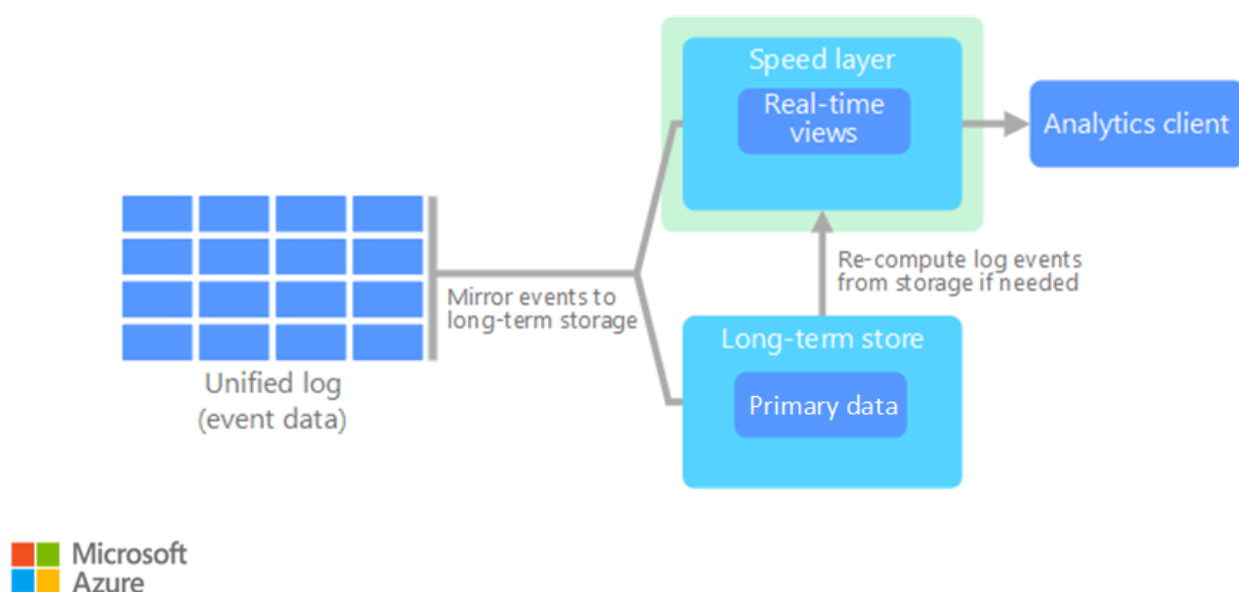
Machine learning in Lambda architecture

Lambda architectures support machine learning workloads by providing both historical data for model training and real-time data for inference. The batch layer enables training on comprehensive historical datasets using [Azure Machine Learning](#) or Fabric Data Science workloads. The speed layer facilitates real-time model inference and scoring. This dual approach allows for models trained on complete historical data while providing immediate predictions on incoming data streams.

Kappa architecture

A drawback to the Lambda architecture is its complexity. Processing logic appears in two different places, the cold and hot paths, via different frameworks. This process leads to duplicate computation logic and complex management of the architecture for both paths.

The **Kappa architecture** is an alternative to the Lambda architecture. It has the same basic goals as the Lambda architecture, but all data flows through a single path via a stream processing system.



Similar to the Lambda architecture's batch layer, the event data is immutable and all of it is collected, instead of a subset of data. The data is ingested as a stream of events into a distributed, fault-tolerant unified log. These events are ordered, and the current state of an event is changed only by a new event being appended. Similar to the Lambda architecture's speed layer, all event processing is performed on the input stream and persisted as a real-time view.

If you need to recompute the entire dataset (equivalent to what the batch layer does in the Lambda architecture), you can replay the stream. This process typically uses parallelism to

complete the computation in a timely fashion.

Machine learning in Kappa architecture

Kappa architectures enable unified machine learning workflows by processing all data through a single streaming pipeline. This approach simplifies model deployment and maintenance since the same processing logic applies to both historical and real-time data. You can use Azure Machine Learning or Fabric Data Science workloads to build models that process streaming data, enabling continuous learning and real-time adaptation. The architecture supports online learning algorithms that update models incrementally as new data arrives.

Lakehouse architecture

A data lake is a centralized data repository that stores structured data (database tables), semi-structured data (XML files), and unstructured data (images and audio files). This data is in its raw, original format and doesn't require predefined schema. A data lake can handle large volumes of data, so it's suitable for big data processing and analytics. Data lakes use low-cost storage solutions, which provide a cost-effective way to store large amounts of data.

A data warehouse is a centralized repository that stores structured and semi-structured data for reporting, analysis, and BI purposes. Data warehouses can help you make informed decisions by providing a consistent and comprehensive view of your data.

The **Lakehouse architecture** combines the best elements of data lakes and data warehouses. The pattern aims to provide a unified platform that supports both structured and unstructured data, which enables efficient data management and analytics. These systems typically use low-cost cloud storage in open formats, such as Parquet or Optimized Row Columnar, to store both raw and processed data.



Common use cases for a lakehouse architecture include:

- **Unified analytics:** Ideal for organizations that need a single platform for both historical and real-time data analysis
- **Data governance:** Ensures compliance and data quality across large datasets

Machine learning in Lakehouse architecture

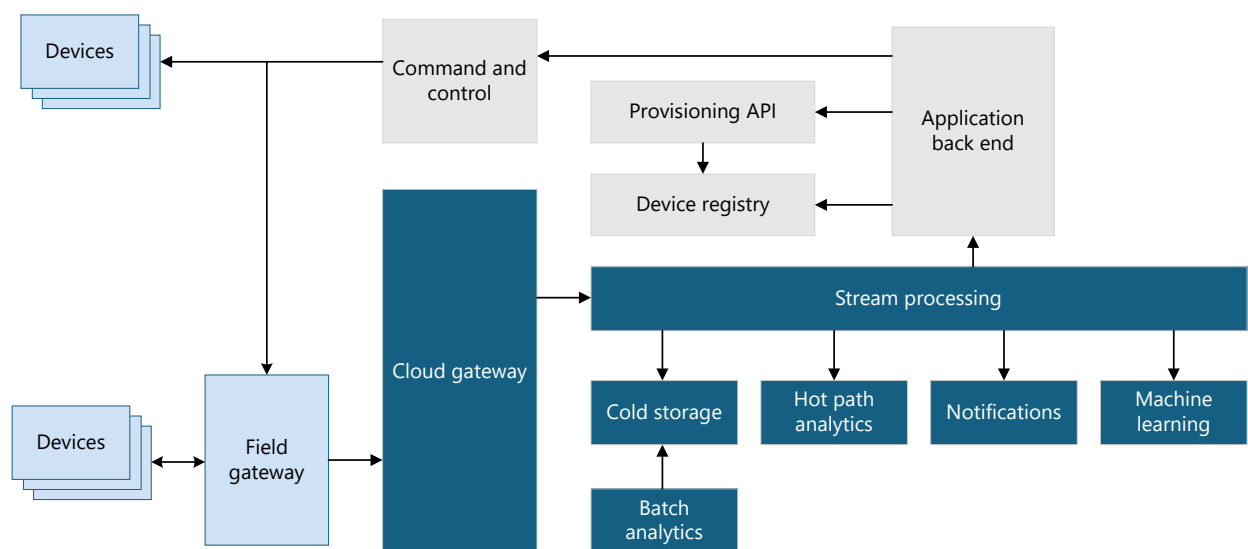
Lakehouse architectures excel at supporting end-to-end machine learning workflows by providing unified access to both structured and unstructured data. Data scientists can use Fabric Data Science workloads to access raw data for exploratory analysis, feature engineering, and model training without complex data movement. The architecture supports the complete machine learning lifecycle, from data preparation and model development using Azure Machine Learning or Fabric notebooks, to model deployment and monitoring. The unified storage layer enables efficient collaboration between data engineers and data scientists while maintaining data lineage and governance.

IoT

The IoT represents any device that connects to the internet and sends or receives data. IoT devices include PCs, mobile phones, smart watches, smart thermostats, smart refrigerators, connected automobiles, and heart monitoring implants.

The number of connected devices grows every day, and so does the amount of data that they generate. This data is often collected in environments that have significant constraints and sometimes high latency. In other cases, thousands or millions of devices send data from low-latency environments, which requires rapid ingestion and processing. You must properly plan to handle these constraints and unique requirements.

Event-driven architectures are central to IoT solutions. The following diagram shows a logical architecture for IoT. The diagram emphasizes the event-streaming components of the architecture.



The **cloud gateway** ingests device events at the cloud boundary via a reliable, low-latency messaging system.

Devices might send events directly to the cloud gateway or through a **field gateway**. A field gateway is a specialized device or software, usually collocated with the devices, that receives events and forwards them to the cloud gateway. The field gateway might also preprocess the raw device events, which includes performing filtering, aggregation, or protocol transformation functions.

After ingestion, events go through one or more **stream processors** that can route the data to destinations, such as storage, or perform analytics and other processing.

Common types of processing include:

- Writing event data to cold storage for archiving or batch analytics.
- Hot path analytics. Analyze the eventstream in near real time to detect anomalies, recognize patterns over rolling time windows, or trigger alerts when a specific condition occurs in the stream.
- Handling special types of nontelemetry messages from devices, such as notifications and alarms.
- Machine learning for predictive maintenance, anomaly detection, and intelligent decision-making.

In the previous diagram, the gray boxes are components of an IoT system that aren't directly related to event streaming. They're included in the diagram for completeness.

- The **device registry** is a database of the provisioned devices, including the device IDs and usually device metadata, such as location.
- The **provisioning API** is a common external interface for provisioning and registering new devices.
- Some IoT solutions allow **command and control messages** to be sent to devices.

Machine learning in IoT architecture

IoT architectures use machine learning for intelligent edge computing and cloud-based analytics. Edge devices can run lightweight models for real-time decision-making, while comprehensive models process aggregated data in the cloud using Azure Machine Learning or Fabric Data

Science workloads. Common applications include predictive maintenance, anomaly detection, and automated response systems. The architecture supports both streaming analytics for immediate insights and batch processing for model training and refinement using historical IoT data.

Next steps

- [IoT Hub](#)
- [Azure Data Explorer](#)
- [Microsoft Fabric decision guide: Choose a Data store](#)
- [Azure Databricks](#)
- [Azure Machine Learning](#)
- [Fabric Data Science](#)

Related resources

- [IoT architectures](#)
- [Big data architecture style](#)
- [Modern analytics architecture with Azure Databricks](#)
- [Big data architectures with Fabric](#)

Last updated on 09/30/2025